Inside the walk block we want all the methods to have turtle as the receiver, but they have self as the implicit receiver. And as written, self is based on the class or location where this whole block is defined, and that is not the object named turtle.

So, what is the correct implementation of walk? Well, we clearly have to use instance_eval, because we want the DSL commands in the block to call the methods in the turtle object. We also have to arrange to pass the block given to the walk method to be evaluated by that instance_eval call. Our implementation looks like this:

```
def walk(&)
  instance_eval(&)
end
```

Notice how we use Ruby's anonymous block syntax to grab the block passed to walk and pass it as-is as a block to instance_eval.

Is this a good use of instance_eval? It depends on the circumstances. The benefit is that the code inside the block looks simple—you don't have to make the receiver explicit:

```
4.times do
  turtle.forward(4)
  turtle.left
end
```

There's a drawback, though. Inside the block, scope isn't what you might think it is, so this code wouldn't work:

```
@size = 4
turtle.walk do
  4.times do
    turtle.forward(@size)
    turtle.left
  end
end
```

Instance variables are looked up in self, and self in the block isn't the same as self in the code that sets the instance variable @size. Because of this kind of confusion, you may want to move away from this style of instance_evaled block.

Using Hook Methods

In Class Methods and Modules, on page 392, we defined a method called included in our GeneralLogger module. When this module was included in a class, Ruby automatically invoked this included method, allowing our module to add class methods to the host class.

included is an example of a *hook method* (sometimes called a *callback*). A hook method is a method that you write but that Ruby calls from within the interpreter when some particular event occurs. The interpreter looks for these methods by name—if you define a method in the right context with an appropriate name, Ruby will call it when the corresponding event happens.

The methods that can be invoked from within the interpreter are: